

Basic (typed function + custom error) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/custom_errors_practice.py`, under its `# Basic` section — the same file also holds the [Failure handling exercise](#), under a `# Failure handling` section. Run it with `cd practice && python custom_errors_practice.py`.

Used later by: the [Real-world wiring exercise](#) and the [Build Task](#). Neither imports this file — they re-write the same one-line custom-error pattern as `MissingConfigError` in `practice/config_logging_wiring/exceptions.py` and `practice/build_task/exceptions.py`.

Basic Version

Approach 1 — the direct way

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    pass

def set_age(age):
    if age < 0:
        raise InvalidAgeError("age cannot be negative: " + str(age))

# test it
set_age(25)  # this works fine, nothing happens

try:
    set_age(-5)
except InvalidAgeError as e:
    print("Caught it:", e)
```

Expected output:

```
Caught it: age cannot be negative: -5
```

This version works correctly. It's missing type hints, and it builds the message with `+` instead of an f-string — both fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — f-string message

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    pass

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(f"age cannot be negative: {age}")

set_age(25) # runs fine, no output

try:
    set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e}")
```

Expected output:

```
Caught it: age cannot be negative: -5
```

Approach 2 — a guard clause with an early return check, and a docstring

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    """Raised when an age value is negative."""
    pass

def set_age(age: int) -> None:
    """Validate an age. Raises InvalidAgeError if age is negative."""
    if age >= 0:
        return
    raise InvalidAgeError(f"age cannot be negative: {age}")
```

```
set_age(25)

try:
    set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e}")
```

Expected output:

```
Caught it: age cannot be negative: -5
```

Difference from Basic: both Intermediate approaches add full type hints — `age: int , -> None` — turning the function's signature into documentation a reader (or an editor's autocomplete) can rely on without opening the function body. Approach 2 also adds docstrings and flips the condition into an early-return guard clause, which some style guides prefer because it keeps the "normal path" unindented and the error path clearly separate — purely a style choice, not a behavior difference from Approach 1.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the exception carries its own data

```
# practice/custom_errors_practice.py – Basic section
class InvalidAgeError(Exception):
    def __init__(self, age: int) -> None:
        self.age = age
        super().__init__(f"age cannot be negative: {age}")

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(age)

set_age(25) # runs fine, no output

try:
    set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e}")
    print(f"The bad value was: {e.age}")
```

Expected output:

```
Caught it: age cannot be negative: -5
The bad value was: -5
```

Approach 2 — a base `ValidationError` other checks can share

```
# practice/custom_errors_practice.py – Basic section
class ValidationError(Exception):
    """Base class for any input-validation problem in this module."""
    pass

class InvalidAgeError(ValidationError):
```

```

def __init__(self, age: int) -> None:
    self.age = age
    super().__init__(f"age cannot be negative: {age}")

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(age)

try:
    set_age(-5)
except ValidationError as e:          # catches InvalidAgeError too, via the base
    print(f"Validation failed: {e}")

```

Expected output:

```
Validation failed: age cannot be negative: -5
```

Approach 3 — a dataclass-based exception with a `__post_init__` check (production style)

```

# practice/custom_errors_practice.py – Basic section
from dataclasses import dataclass, field

class ValidationError(Exception):
    pass

@dataclass
class InvalidAgeError(ValidationError):
    age: int
    message: str = field(init=False)

    def __post_init__(self) -> None:
        self.message = f"age cannot be negative: {self.age}"
        super().__init__(self.message)

def set_age(age: int) -> None:
    if age < 0:
        raise InvalidAgeError(age)

try:

```

```

set_age(-5)
except InvalidAgeError as e:
    print(f"Caught it: {e.message}")
    print(f"As a dict-like repr: {e!r}")

```

Expected output:

```

Caught it: age cannot be negative: -5
As a dict-like repr: InvalidAgeError(age=-5, message='age cannot be negative: -5')

```

Difference from Intermediate, and between these 3 Advanced approaches: Intermediate's exception is a bare name with a fixed message — useful to a human reading logs, useless to code that wants to react to *which* value was wrong. Approach 1 fixes that by storing `self.age`. Approach 2 additionally makes the exception part of a small hierarchy (`ValidationError` as a shared base), so a caller who doesn't care about the specific validation failure can catch the whole family at once — useful the moment you have more than one kind of validation error in the same module (age, email, phone number, ...) and want one broad `except ValidationError` alongside specific ones where it matters. Approach 3 is the most "production" of the three: `@dataclass` auto-generates `__init__`, `__repr__`, and equality comparison for you, which matters once exceptions get logged as structured data (e.g. to a JSON log) rather than just printed as text — `repr(e)` becomes genuinely inspectable instead of just a sentence.

Which one should you actually write? For an exercise or a small script, Intermediate Approach 1 (f-string, type hints, no extra machinery) is completely correct and what most people should default to. Advanced Approach 1 (exception carries its own data) is worth the extra few lines the moment *any* caller needs to react to the specific bad value, not just log it. Advanced Approach 2's shared base class is worth it once you have 2+ related exception types in the same file. Advanced Approach 3's dataclass style is worth it specifically in codebases with structured logging — don't reach for it by default, it's solving a problem ("I need this exception to serialize cleanly") that a simple script usually doesn't have.